Problem statement- assignment 2

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

Ans1.

Domain	Self-Rating	Justification & Current Capability
Machine Learning (ML)	**B** *(Can code under supervision)*	Strong knowledge of the entire ML workflow, including data pre-processing, exploratory data analysis (EDA), feature engineering, and model evaluation. I have worked with algorithms such as **Random Forest, Extra Trees, CatBoost, SVM, and KNN** using Scikit-Learn, Pandas, and Matplotlib. While I do not write complex pipelines entirely from scratch from memory, I can effectively build, debug, and customize ML models using documentation and technical supervision.
Artificial Intelligence (AI)	**B** *(Can code under supervision)*	Excellent understanding of fundamental AI ideas, problem-solving heuristics, and search /optimization methodologies. I have used **Metaheuristic Feature Selection techniques** (e.g., Grasshopper Optimization, Moth Flame Optimization, Multi-Verse Optimizer and Sequential Feature Selection) in academic research projects and can implement AI workflows with guidance from senior architecture and reference materials.
Deep Learning (DL)	**B** *(Can code under supervision)*	Good theoretical knowledge of neural networks: layers, weights, activation functions, forward/backward propagation and **Multi-Layer Perceptrons** (MLP). I have used MLP classifiers in model comparisons and I can implement neural network scripts

Domain	Self-Rating	Justification & Current Capability
		under supervision rather than write complex architectures from scratch.
Large Language Models (LLM)	**B / C** *(Conceptual understanding; can integrate APIs under supervision)*	I understand the basic working principles of LLMs (tokenization, embeddings, context windows, Prompt Engineering, and RAG architecture) and have experience working with REST APIs in Python. I haven't built or fine-tuned LLMs from scratch but I can help implement LLM APIs and build workflows under close supervision.

2. What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level.

Ans 2. Architecture components to create a chatbot based on LLM

Pipeline Phase	Step & Architectural Component	Input ➔ Output Flow	High-Level Role in the Chatbot
Phase 1: Offline Ingestion	**1. Document Loader & Chunker**	Raw Docs ➔ Text Chunks	It reads company PDFs/files and breaks long text into smaller chunks of 500-1000 tokens with a little overlap.
	2. Embedding & Vector Store	Text Chunks ➔ Indexed Vectors	Converts text chunks into numerical arrays (embeddings) and indexes them inside the Vector Database.
Phase 2: Online Query Flow	**3. UI, API Gateway & Guardrails**	User Prompt ➔ Validated Query	Captures user input, handles authentication, and blocks unsafe prompts or prompt-injection attacks.

Pipeline Phase	Step & Architectural Component	Input ➔ Output Flow	High-Level Role in the Chatbot
	4. Semantic Retriever	Query Vector ➔ Top-K Context	Converts the user's question to a vector and finds the most relevant paragraphs from the Vector DB.
	5. Memory & Prompt Builder	Context + History ➔ Final Prompt	Combines user query, retrieved Vector DB context, and previous conversation history into a structured prompt.
	6. LLM Inference Engine	Final Prompt ➔ Chatbot Reply	Sends the enriched prompt to the LLM (e.g., GPT-4, Claude, Llama-3) to generate a grounded, factual answer.

Key Architectural Components & High-Level Approach

(i) User Interface (Frontend): A simple web page or chat window where the user types their question and sees the reply.

(ii) Data Loader & Text Splitter (Chunking): LLMs cannot read huge PDF files all at once because they have a token limit. So, this component loads the documents and breaks them into smaller pieces (for example, 500 words per chunk).

(iii) Embedding Model: Computers do not understand English words directly. An embedding model converts our text chunks into a list of numbers (vectors) based on their meaning.

(iv) Vector Database: A special database that stores those number vectors so we can quickly search for matching information when a user asks a question.

(v) Retriever / Orchestrator (like LangChain): When a user asks a question, the retriever converts the question into a vector, goes to the Vector Database, and pulls out the top 3 or 4 most relevant paragraphs.

(vi) Chat Memory: By default, LLMs forget previous messages. A memory component (using SQLite or Redis) saves the last few messages so the user can ask follow-up questions.

(vii) LLM Engine & Guardrails: Finally, the user's question and the matched paragraphs are sent to the LLM (such as OpenAI GPT, Gemini, or Llama). Basic guardrails are added to make sure the user is not asking harmful questions, and the LLM generates the final human-like response.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

Ans 3. In traditional relational databases like Oracle SQL or SQLite, data is stored in rows and columns, and we search data using exact word matching (for example, WHERE name = 'Miller'). This works well for numbers and fixed text, but it fails when we want to search by meaning. For example, if a document contains the word "Laptop" and the user searches for "Computer", a normal SQL query will not match them.

A Vector Database solves this problem. It stores data as vector embeddings (arrays of numbers like [0.12, -0.45, 0.88...]) created by an AI model. Words or sentences that have similar meanings are stored close to each other in mathematical space. When a user searches for something, the Vector DB calculates the distance between vectors (using simple math methods like Cosine Similarity) and finds the closest match based on meaning rather than exact spelling.

Hypothetical Problem: Company Security & Policy Document Assistant

Problem Definition: Suppose a cybersecurity company has thousands of internal PDF documents, security policies, and past bug/incident reports. Whenever a new engineer joins or an error occurs, it takes hours to manually read through folders to find the solution. We want to build a search system where engineers can type a problem in plain English and get the exact solution from old reports.

We need from the database:

- It should search by meaning (semantic search).
- It should be open-source and self-hosted because company security files are private and cannot be uploaded to a third-party cloud.
- It should be easy to connect with Python.

Comparison of Popular Vector Databases:

Feature	**ChromaDB**	**Qdrant**	**Pinecone**
Type	Open-Source	Open-Source	Closed-Source (Cloud only)
Data Privacy	Can run locally on own server	Can run on own server (Docker)	Data goes to Pinecone's cloud
Ease of Use	Very easy for Python beginners	Moderate	Easy (Managed API)

Feature	ChromaDB	Qdrant	Pinecone
Best Used For	Small to medium projects & prototypes	Large production apps with heavy filtering	Teams who want a ready-made cloud service

For this problem, I would choose ChromaDB for initial development (and Qdrant if the data grows into millions of documents) for the following simple reasons:

(i) Data Privacy (Self-Hosted): Since the problem involves internal security documents, we cannot send private data to an external cloud like Pinecone. Both ChromaDB and Qdrant are open-source and can be run on our own local machine or private server.

(ii) Easy Python Integration: As someone who primarily codes in Python, ChromaDB is very simple to set up just like SQLite, it can run directly inside a Python script (pip install chromadb) without needing complex server setup.

(iii) Metadata Filtering: Along with meaning-based search, they allow simple filtering (for example, filtering documents by year = 2025 or category = 'network'), which helps find accurate security reports faster.

(iv) Cost-Effective: Being open-source and free to use, we can build and test the complete system without paying for cloud subscriptions.